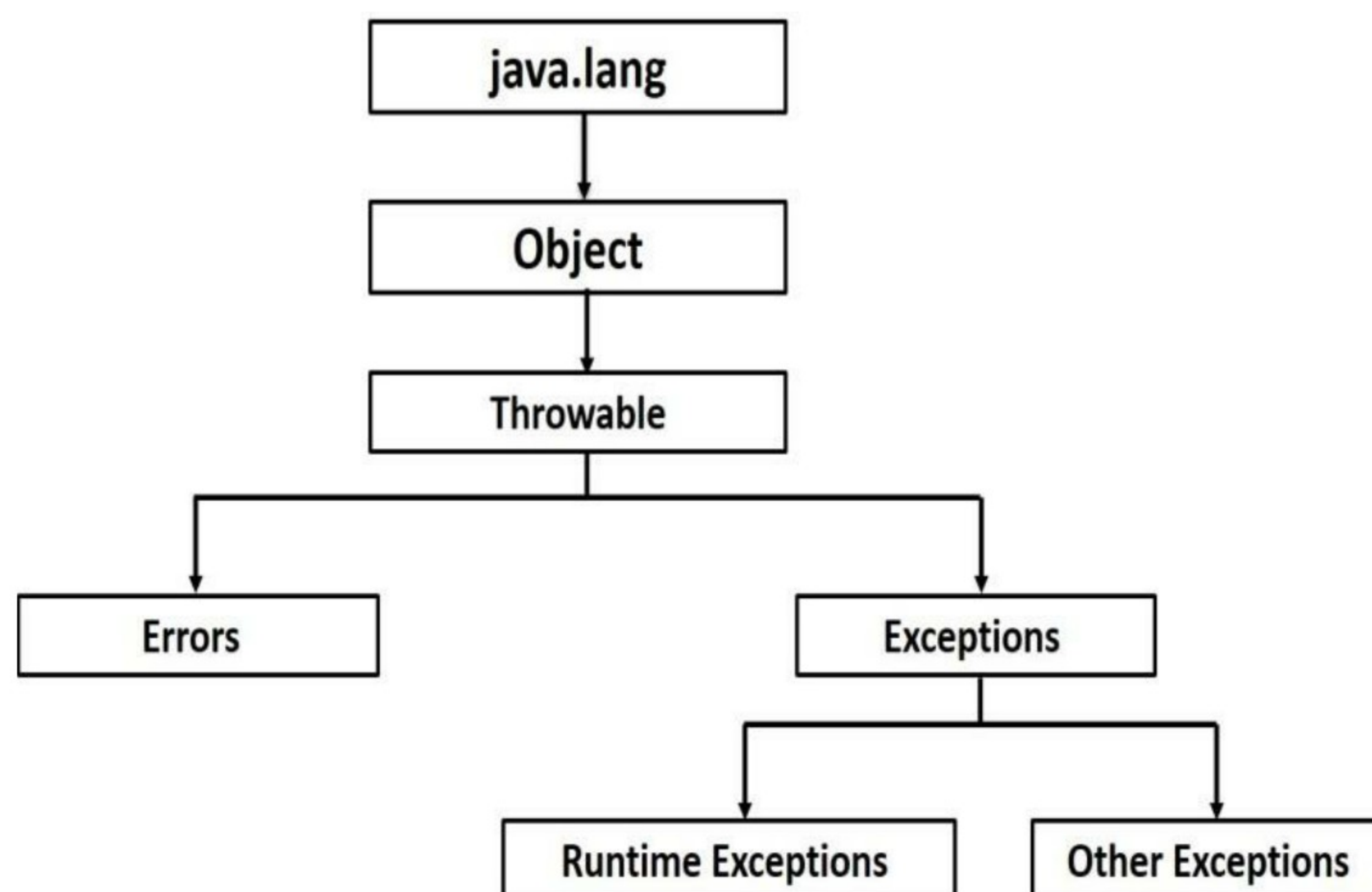


Exceptions Handling

- Exception (અથવા Exception event) એ એક problem છે જે પ્રોગ્રામના execution દરમિયાન generate થાય છે.
- જ્યારે કોઈ Exception generate થાય છે ત્યારે પ્રોગ્રામનો સામાન્ય flow disrupted થાય છે અને પ્રોગ્રામ / એપ્લિકેશન terminate થાય છે, તેથી આ Exception ને handled કરવાની જરૂર છે.
- જાવા માં Exception એ એક object છે જે પ્રોગ્રામમાં થતાં exception ને describes કરે છે. જ્યારે જાવામાં કોઈ exceptional generate થાય છે, ત્યારે exception ને thrown કરવામાં આવે છે.
- Exception generate ઘણા જુદા જુદા કારણોસર થઈ શકે છે, તેમાંથી કેટલાક નીચે મુજબ છે:
 - User એ invalid ડેટા enter કરે ત્યારે છે.
 - જે ફાઈલને open કરવાની જરૂર છે તે શોધી શકાતી નથી.
 - Communications માં નેટવર્ક કનેક્શન lost થઈ ગયું છે, અથવા JVM પાસે પુરતી મેમરી નથી.
- Users, programmer's અથવા physical resources failed થવાથી Exception generate થાય છે.
- જાવામાં Exception Handling એ Exception (runtime errors) ને handle કરવા માટે એક powerful method છે, જેથી એપ્લિકેશનનો normal flow જાળવી શકાય.
- જાવામાં Exceptions ની ત્રણ categories છે:
 - 1) Checked exceptions:** checked exception એ exception છે જે કમ્પાઈલ સમયે આવે છે, આને compile time exceptions પણ કહેવામાં આવે છે. ઉદાહરણ, IOException, SQLException વગેરે.
 - 2) Runtime exceptions:** Unchecked exception એ exception છે જે execution દરમિયાન આવે છે, એને Runtime Exceptions તરીકે પણ ઓળખવામાં આવે છે. આમાં પ્રોગ્રામિંગ bugs હોય છે, જેમ કે logic errors અથવા API નો improper ઉપયોગ.
 - Compilation સમયે રનટાઈમ exception ને ignored કરવામાં આવે છે.
 - Example:** ArithmeticException, NullPointerException, Array Index out of Bound exception.
 - 3) Errors:** આ કોઈ પણ exceptions નથી, પરંતુ problems કે જે user અથવા programmer ની control બહાર છે.
Example: OutOfMemoryError, VirtualMachineErrorException.
- બધા exception classes એ java.lang.Exception class ની subtypes છે. Exception class એ Throwable class નો subclass છે. exception class સિવાય અન્ય એક subclass છે જેને Error કહેવામાં આવે છે જે Throwable class માંથી derived કરવામાં આવ્યો છે.



- Errors સામાન્ય રીતે જાવા પ્રોગ્રામ્સમાં હોતી નથી. આ conditions સામાન્ય રીતે failuresના caseમાં થાય છે, જે જાવા programs દ્વારા handle કરવામાં આવતી નથી. Runtime environment દ્વારા generate થતી Errors સૂચવવા માટે Errors generated થાય છે. ઉદાહરણ: JVM ને મેમરી નથી છે. સામાન્ય રીતે પ્રોગ્રામ્સ errors માંથી recover થઈ શકતા નથી.
- Exception class માં બે main subclasses છે: IOException અને RuntimeException class

Exception Handling Mechanism

- Exception handling પાંચ કીવર્ડ્સનો ઉપયોગ કરીને કરવામાં આવે છે:
 - 1) try
 - 2) catch
 - 3) finally
 - 4) throw
 - 5) throws

Using try and catch

1) try block :

- જાવા માં try block નો ઉપયોગ કોડને લખવામાં થાય છે જે કદાચ exception throw કરી શકે.
- જાવા માં try block પછી કેચ અથવા finally block કરવું જરૂરી છે.

2) catch block:

- Exceptionને હેન્ડલ કરવા માટે catch block નો ઉપયોગ થાય છે. તેનો ઉપયોગ ફક્ત try block પછી જ થવો જોઈએ.
- કેચ બ્લોકને checked કરવામાં આવે છે, જો કે જે exception આવે છે તે type કેચ બ્લોકમાં આપવામાં આવે છે, તો exception તે કેચ બ્લોકને સોંપવામાં આવે છે.
- તમે single try ની સાથે multiple catch blockનો ઉપયોગ કરી શકો છો.

Syntax:

```
try
{
    //Protected code
}
catch(ExceptionName1 e1)
{
    //Catch block 1
}
catch(ExceptionName2 e2)
{
    //Catch block 2
}
```

- ઉપરની syntaxમાં, ત્યાં બે કેચ બ્લોક્સ છે. દ્વાર બ્લોકમાં, કોડ લખીએ છીએ જે exception generate કરે છે. જો protected code દ્વારા generate થયેલ exception ને first catch block સાથે match કરવામાં આવશે.
- જો thrown exception નો ડેટા type ExceptionName1 સાથે match થાય છે, તો તે ત્યાં catch થઈ જાય છે અને કેચ બ્લોક execute થાય છે.
- જો નહીં, તો exception ને second કેચ બ્લોકમાં check કરવામાં આવે છે.
- exception catch થાય ત્યાં સુધી અથવા તમામ catch check ન થાય ત્યાં સુધી ચાલુ રહે છે, તે case માં current Method નું execution અટકાવે છે.

Example:

```
class demoTry
{
    public static void main(String[] args)
    {
        try
        {
            int arr[]={1,2,3};
            arr[3]=3/0;
        }
        catch(ArithmeticException ae)
        {
            System.out.println("Divide by zero :: " + ae);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Array index out of bound exception :: "+e);
        }
    }
}
```

Output:

divide by zero :: java.lang.ArithmeticException: / by zero

Nested Try-Catch Blocks

- જાવામાં, try block use બીજા try block ની અંદર કરવામાં આવે તેને નેસ્ટેડ ટ્રાય બ્લોક તરીકે ઓળખવામાં આવે છે.
- નેસ્ટેડ try block નો ઉપયોગ ત્યારે થાય છે જ્યારે બ્લોકના એક ભાગમાં એક error આવે છે ત્યારે આખા બ્લોકમાં બીજી error generate થાય છે.
- જો અંદરના try blockમાં કોઈ particular exception માટે કેચ હેન્ડલર નથી, તો બહારના try ના catch ને check કરવામાં આવે છે.
- આ checking કેચ સ્ટેટમેન્ટ્સમાંથી એક match થાય ત્યાં સુધી ચાલુ રહે છે, અથવા ત્યાં સુધી entire નેસ્ટેડ ટ્રાય સ્ટેટમેન્ટ્સ પૂર્ણ ન થાય ત્યાં સુધી. જો કોઈ કેચ સ્ટેટમેન્ટ્સ મેચ થતું નથી, તો જાવા રન-ટાઈમ સિસ્ટમ exceptionને હેન્ડલ કરે છે.

Syntax:

```
try
{
    Statement 1;
    try
    {
        //Protected code
    }
    catch(ExceptionName e1)
    {
        //Catch block1
    }
}
catch(ExceptionName1 e2)
```

```
{  
    //Catch block 2  
}
```

Example:

```
class demoTry1  
{  
    public static void main(String[] args)  
    {  
        try  
        {  
            int arr[]={5,0,1,2};  
            try  
            {  
                arr[4] = arr[3]/arr[1];  
            }  
            catch(ArithmeticException e)  
            {  
                System.out.println("divide by zero :: "+e);  
            }  
        }  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("array index out of bound exception :: "+e);  
        }  
        catch(Exception e)  
        {  
            System.out.println("Generic exception :: "+e);  
        }  
        System.out.println("Out of try..catch block");  
    }  
}
```

Output:

```
divide by zero :: java.lang.ArithmeticException: / by zero  
Out of try..catch block
```

3) finally:

- Finally keywordનો ઉપયોગ કોડનો બ્લોક બનાવવા માટે થાય છે જે try અથવા catch block ને follow કરે છે.
- Finally block હંમેશાં execute થાય છે exception આવે કે નહીં.
- કેચ બ્લોક પછી માં finally block આવે છે.

Syntax:

```
try  
{  
    //Protected code  
}  
catch(ExceptionType1 e1)  
{
```

```
        //Catch block 1
    }
    catch(ExceptionType2 e2)
    {
        //Catch block 2
    }
    finally
    {
        //The finally block always executes.
    }
```

Example:

```
class demoFinally
{
    public static void main(String args[])
    {
        int a[] = new int[2];
        try
        {
            System.out.println("Access element three :" + a[3]);
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("Exception thrown :" + e);
        }
        finally
        {
            a[0] = 10;
            System.out.println("First element value: " +a[0]);
            System.out.println("The finally block is always executed");
        }
        System.out.println("Out of try...catch...finally... ");
    }
}
```

Output:

```
Exception thrown :java.lang.ArrayIndexOutOfBoundsException: 3
First element value: 10
The finally block is always executed
Out of try...catch...finally...
```

Key points to keep in mind:

- try statement વગર catch block use થઈ શકે નહિ.
- દરેક try/catch માટે finally block હોવો ફરજિયાત નથી.
- try block એ catch block અથવા finally block વગર use થઈ શકે નહિ.

- કોઈપણ કોડ try, catch, finally blocks ની વચ્ચે present હોઈ શકતો નથી.

4) throw :

- throw keyword નો ઉપયોગ explicitly exception ને throw કરવા માટે થાય છે.
- throw કીવર્ડનો ઉપયોગ કરીને આપણે કાં તો checked અથવા unchecked exception throw કરી શકીએ.
- ફક્ત Throwable class અથવા તેના subclasses નો object ને thrown કરી શકાય છે.
- પ્રોગ્રામ નું એક્ઝેક્યુશન થો સ્ટેટમેન્ટ પર અટકે છે, અને closest catch statement ને exception માટે match કરવામાં આવે છે.

Syntax:

throw **ThrowableInstance**;

Example:

```
class demoThrow
{
    static void demo()
    {
        try
        {
            throw new ArithmeticException("demo");
        }
        catch(ArithmeticException e)
        {
            System.out.println("Exception caught");
        }
    }
    public static void main(String args[])
    {
        demo();
    }
}
```

Output:

Exception caught

5) throws:

- throws keywordનો ઉપયોગ exception ને declare કરવા માટે થાય છે
- જો કોઈ method એ checked exceptionને હેન્ડલ કરતી નથી, તો method એ throws keywordનો ઉપયોગ કરીને તેને declare કરવી જોઈએ. throws keyword એ method's signature ની અંતમાં આવે છે.
- તમે multiple exceptions declare કરી શકો છો.

Syntax:

```
return_type method_name() throws exception_class_name_list
{
    //method code
}
```

Example:

```
class demoThrows
```

```
{
    static void display() throws ArithmeticException
    {
        System.out.println("Inside check function");
        throw new ArithmeticException("Demo");
    }
    public static void main(String args[])
    {
        try
        {
            display();
        }
        catch(ArithmeticException e)
        {
            System.out.println("caught :: " + e);
        }
    }
}
```

Output:

```
Inside check function
caught :: java.lang.ArithmeticException: Demo
```

User Defined Exception

- જાવામાં, આપણે આપણું પોતાનું exception બનાવી શકીએ જે કસ્ટમ exception અથવા user-defined exception તરીકે ઓળખાય છે.
- આપણો પોતાનો exception અને message હોઈ શકે છે.

Key points to keep in mind:

- બધા exceptions એ Throwable class ની child હોવા જરૂરી છે.
- જો તમે checked exception લખવા માંગતા હોવ કે જે Declare Rule દ્વારા automatically લાગુ કરવામાં આવે છે, તો તમારે Exception class extend કરવાની જરૂર છે.
- જો તમે runtime exception ને write કરવા માંગતા હો, તો તમારે RuntimeException class ને extend કરવાની જરૂર છે.

Example:

```
class demoUserException extends Exception
{
    private int ex;
    demoUserException(int a)
    {
        ex=a;
    }
    public String toString()
    {
        return "MyException[" + ex + "] is less than zero";
    }
}
```

```

}
class demoException
{
    static void sum(int a,int b) throws demoUserException
    {
        if(a<0)
        {
            throw new demoUserException (a);
        }
        else
        {
            System.out.println(a+b);
        }
    }
    public static void main(String[] args)
    {
        try
        {
            sum(-10, 10);
        }
        catch(demoUserException e)
        {
            System.out.println(e);
        }
    }
}

```

Output:

MyException[-10] is less than zero

List of Java Exception (Built-In Exception)

- જાવા standard પેકેજ java.lang અંદર ઘણા બિલ્ટ-ઇન exception classes ને defines કરે છે.

Checked Exception:

Exception	Description
ClassNotFoundException	આ Exception ત્યારે આવે છે જ્યારે જાવા રન-ટાઇમ સિસ્ટમ પ્રોગ્રામમાં specified class ને શોધવામાં નિષ્ફળ જાય છે.
IllegalAccessException	આ Exception ત્યારે આવે છે જ્યારે તમે કોઈ abstract class અને interfaceનો object બનાવો છો.
NoSuchMethodException	આ Exception ત્યારે આવે છે જ્યારે તમે call કરો છો તે method class માં અસ્તિત્વમાં નથી.
NoSuchFieldException	requested કરેલ field અસ્તિત્વમાં નથી.

Unchecked Exception:

Exception	Description
ArithmeticException	આ Exception આવે છે, જ્યારે તમે કોઈ સંખ્યાને શૂન્યથી divide કરો છો ત્યારે

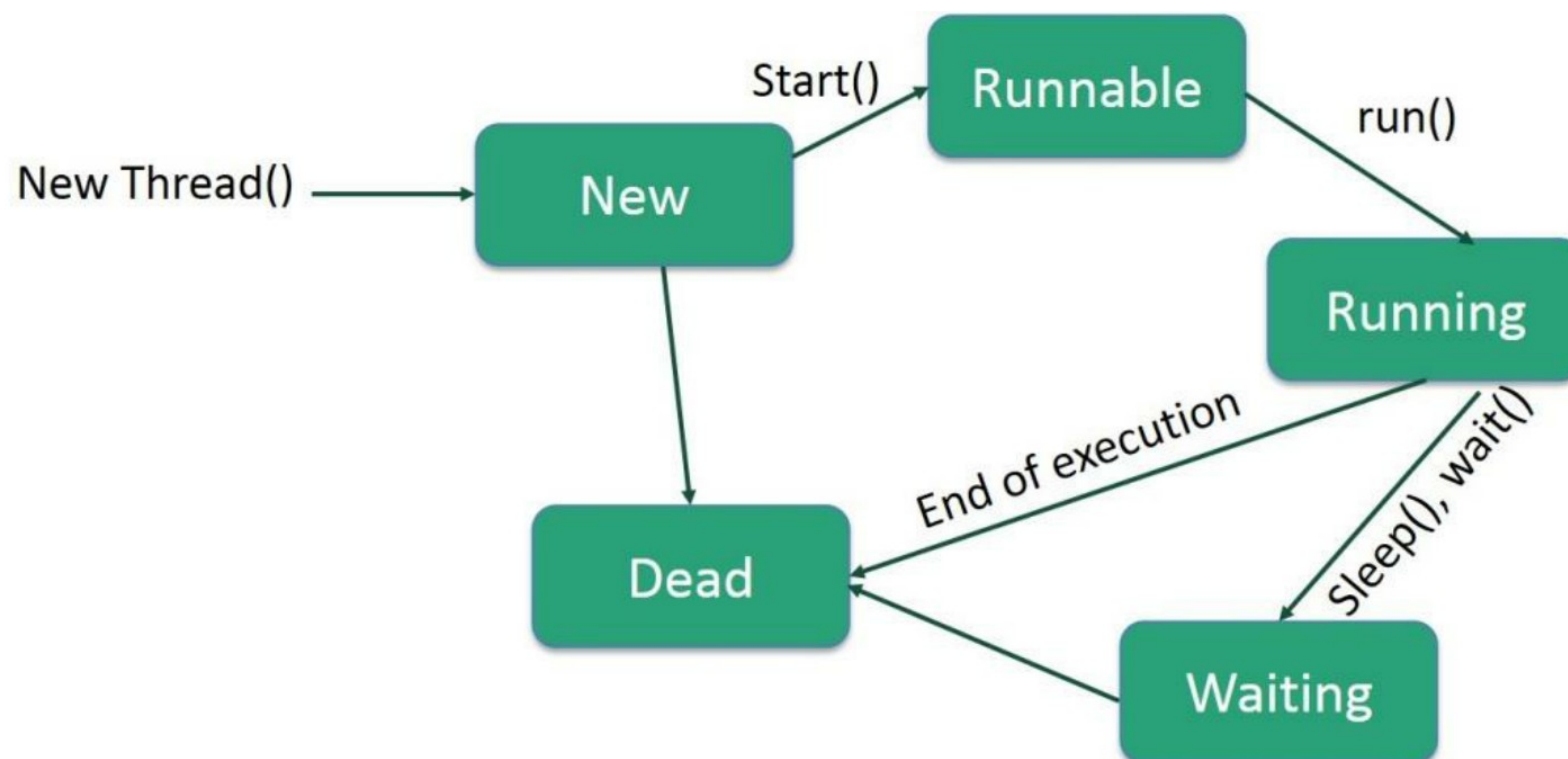
	Arithmetic Exception આવે છે.
ArrayIndexOutOfBoundsException	આ Exception આવે છે, જ્યારે તમે કોઈ array ની size કરતા વધારે index થી array ની element ને access કરો છો.
NumberFormatException	આ Exception આવે છે, જ્યારે તમે ખોટા format વાળા string variable ને integer (numeric forma) માં convert કરવાનો પ્રયાસ કરો છો જે એકબીજા સાથે compatible નથી.
ClassCastException	Invalid cast.
NullPointerException	null reference નો Invalid ઉપયોગ.

Multithreading

- જાવા એ મલ્ટિથ્રેડ પ્રોગ્રામિંગ લેંગ્વેજ છે, જેનો અર્થ છે કે આપણે જાવાનો ઉપયોગ કરીને મલ્ટિ થ્રેડ પ્રોગ્રામ બનાવી શકીએ.
- મલ્ટિથ્રેડ પ્રોગ્રામ્સમાં બે અથવા વધુ થ્રેડો હોય છે જે એક સાથે run થઈ શકે છે. આનો અર્થ એ છે કે એક જ પ્રોગ્રામ એક સાથે બે અથવા વધુ કાર્યો કરી શકે છે.
- થ્રેડ એ lightweight sub-process છે, જે processing નો સૌથી small unit છે.
- મલ્ટિપ્રોસેસિંગ અને મલ્ટિથ્રેડિંગ, બંનેનો ઉપયોગ મલ્ટિટાસ્કિંગ કરવા માટે થાય છે.
- પરંતુ મલ્ટિપ્રોસેસિંગ કરતા મલ્ટિથ્રેડિંગનો ઉપયોગ કરીએ છીએ કારણ કે થ્રેડો common memory area ને share કરે છે. તેઓ અલગ memory area ને allocate કરતા નથી તે મેમરીને બચાવે છે, અને થ્રેડો વચ્ચે context-switching એ process કરતા ઓછો સમય લે છે.
- Threads independent છે. તેથી, જો કોઈ એક થ્રેડમાં exception આવે તો તે અન્ય થ્રેડોને અસર કરતું નથી.
- જાવા મલ્ટિથ્રેડિંગનો ઉપયોગ મોટે ભાગે games, એનિમેશન વગેરેમાં થાય છે.
- ઉદાહરણ તરીકે, એક થ્રેડ ફાઈલ પર content લખી રહ્યો છે તે જ સમયે બીજો થ્રેડ spelling check કરી રહ્યો છે.
- મલ્ટિપ્રોસેસિંગમાં, દરેક process ને મેમરીમાં પોતાનું address હોય છે. તેથી, દરેક process ને અલગ મેમરી area allocates થાય છે.
- એક processમાંથી બીજી process માં સ્વિચ કરવા માટે રજિસ્ટર, મેમરી maps, updating lists વગેરેને કરવા માટે સમય જરૂરી છે તેથી processes વચ્ચેના communication નો cost વધુ છે.
- Disadvantage: જો તમે ઘણા બધા થ્રેડ બનાવો છો, તો તમે ખરેખર તમારા પ્રોગ્રામના performance વધારવાને બદલે degrade કરો છો.
- ઓવરહેડ context switching સાથે સંકળાયેલા છે. જો તમે ઘણા બધા થ્રેડ બનાવો છો, તો તમારા પ્રોગ્રામને ચલાવવા કરતાં વધુ CPU સમય context switchingમાં ખર્ચવામાં આવશે.

Life Cycle of Thread

- એક thread તેના life cycle માં વિવિધ stages માંથી પસાર થાય છે. ઉદાહરણ thread એ born, started, runs, and then dies.
- Diagram:



- **New:**
 - એક નવો થ્રેડ New state થી તેની life cycle ની શરૂઆત કરે છે. જ્યાં સુધી પ્રોગ્રામ Start () methodનો ઉપયોગ કરીને થ્રેડ શરૂ ન કરે ત્યાં સુધી તે આ stateમાં રહે છે. તેને born thread તરીકે પણ ઓળખવામાં આવે છે.
- **Runnable:**
 - start () methodની invocation પછી થ્રેડ runnable stateમાં છે, પરંતુ થ્રેડ શેડ્યૂલરે તેને running thread તરીકે પસંદ કર્યો નથી.
 - જો thread scheduler એ select કર્યો હોય તો થ્રેડ running stateમાં છે.
- **Waiting:**
 - કેટલીકવાર થ્રેડ waiting state માં transitions કરે છે જ્યારે થ્રેડ કોઈ કાર્ય કરવા માટે બીજા થ્રેડની રાહ જુએ છે. જ્યારે બીજો થ્રેડ signal આપે છે ત્યારે જ થ્રેડ continue executing state માં પાછો ફરે.
- **Timed waiting:**
 - running થ્રેડ time ની specified interval માટે timed waiting stateમાં જાય છે. જ્યારે તે સમયગાળો સમાપ્ત થાય છે અથવા જ્યારે તે ઇવેન્ટની રાહ જોતા હોય તે આવે છે ત્યારે runnable stateમાં પાછો આવે છે.
- **Terminated:**
 - જ્યારે ચાલી રહેલ થ્રેડ terminated stateમાં પ્રવેશ કરે છે જ્યારે તે ((run() method exits) તેનું કાર્ય પૂર્ણ કરે છે.

Thread Priorities

- દરેક જાવા થ્રેડ ને priority હોય છે જે operating system ને કયા order માં થ્રેડ schedule કરવા તેમાં હેલ્પ કરે છે.
- Priorities એ MIN_PRIORITY (1 ની constant) અને MAX_PRIORITY (10 ની constant) વચ્ચેની રેન્જમાં છે.
- default રૂપે, દરેક થ્રેડને NORM_PRIORITY (5 ની constant) priority આપવામાં આવે છે
- પ્રોગ્રામ માટે higher priority થ્રેડો વધુ important છે અને lower-priority થ્રેડો પહેલાં higher priority થ્રેડો ને પ્રોસેસર સમય allocate કરવો જોઈએ.
- થ્રેડ શેડ્યૂલર મુખ્યત્વે થ્રેડોને શેડ્યૂલ કરવા માટે preemptive અથવા time slicing scheduling નો ઉપયોગ કરે છે.

Creating a Thread

- થ્રેડ ક્લાસ થ્રેડ પર create અને operations માટે કન્સ્ટ્રક્ટર અને methods provide કરે છે.

1) Constructor of Thread class:

- Thread ()
- Thread (String name)
- Thread (Runnable r)

- Thread (Runnable r, String name)

2) Methods of Thread Class:

Method	Description
public void run()	થ્રેડ માટે Entry point
public void start()	run() મેથડને call કરીને થ્રેડ શરૂ કરી શકો છો.
public String getName()	થ્રેડનું નામ આપે છે.
public void setName(String name)	થ્રેડને નામ આપવું.
public int getPriority()	થ્રેડની priority આપે છે.
public int setPriority(int priority)	આ થ્રેડ object ની priority સેટ કરે છે. possible values 1 અને 10 ની વચ્ચે છે.
public final boolean isAlive()	થ્રેડ હજી ચાલુ છે કે નહીં તે Checks કરે છે.
public static void sleep(long millisec)	specified સમય માટે થ્રેડને Suspend કરે છે
public final void join(long millisec)	થ્રેડ end થાય ત્યાં સુધી રાહ જુઓ.

- જાવા બે રીતે defines કરે છે જેના દ્વારા થ્રેડ બનાવી શકાય છે.

- By implementing the **Runnable** interface.
- By extending the **Thread** class.

1) By implementing the Runnable interface:

- થ્રેડ બનાવવાનો સૌથી સહેલો રસ્તો class બનાવવાનો છે જે runnable interface ને implements કરે છે.
- runnable interface ને implementing કર્યા પછી, class ને run () method ને implement કરવાની હોય છે, જે નીચે આપેલ ફોર્મ ધરાવે છે:

public void run()

- આ method થ્રેડ માટે entry point provides કરે છે અને તમે આ method ની અંદર તમારું business logic ને લખી શકો છો.
- તે પછી, તમે નીચેના કન્સ્ટ્રક્ટરનો ઉપયોગ કરીને થ્રેડ object નો ઈન્સ્ટન્ટ બનાવશો:

Thread (Runnable threadObj, String threadName) ;

- જ્યાં, threadObj એ class નો instance છે જે Runnable interface ને implements કરે છે અને threadName એ નવા થ્રેડને આપેલ નામ છે.
- એકવાર થ્રેડ object બનાવવામાં આવે, પછી તમે start () method ને call કરીને થ્રેડ start કરી શકો છો, જે run () method ને executes કરે છે

void start ();

Example:

```
class demoThread3 implements Runnable
{
    public void run()
    {
        System.out.println("Thread is running...");
    }
}
```

```
public static void main(String args[])
{
    demoThread3 d1=new demoThread3 ();
    Thread t1 =new Thread(d1);
    t1.start();
}
}
```

Output:

Thread is running...

Example:

```
class RunnableDemo implements Runnable
{
    Thread t;
    String threadName;
    RunnableDemo( String name)
    {
        threadName = name;
        System.out.println("Creating " + threadName );
    }
    public void run()
    {
        System.out.println("Running " + threadName );
        try
        {
            for(int i = 2; i >= 0; i--)
            {
                System.out.println("Thread: " + threadName + ", " + i);
                // Let the thread sleep for a while.
                Thread.sleep(1000);
            }
        }
        catch (InterruptedException e)
        {
            System.out.println("Thread " + threadName + " interrupted.");
        }
        System.out.println("Thread " + threadName + " exiting.");
    }
    public void start ()
    {
        System.out.println("Starting " + threadName );
        if (t == null)
        {
            t = new Thread (this, threadName);
            t.start ();
        }
    }
}
```

```
}  
public class ThreadDemo  
{  
    public static void main(String args[])  
    {  
        RunnableDemo R1 = new RunnableDemo( "Thread-1");  
        R1.start();  
    }  
}
```

Output:

Creating Thread-1
Starting Thread-1
Running Thread-1
Thread: Thread-1, 2
Thread: Thread-1, 1
Thread: Thread-1, 0
Thread Thread-1 exiting.

2) By extending the Thread class.

- થ્રેડ બનાવવાની બીજી રીત એ છે કે નવો class બનાવવો જે Thread ક્લાસને extends કરે અને પછી તે class નો instance બનાવવો.
- Extending class એ run() method ને ઓવરરાઈડ કરવી જરૂરી છે, જે નવા થ્રેડ માટે entry point છે.
- એકવાર થ્રેડ object બનાવવામાં આવે, પછી તમે start () method ને call કરીને થ્રેડ start કરી શકો છો, જે run() method ને executes કરે છે

Example:

```
class demoThread2 extends Thread  
{  
    public void run()  
    {  
        System.out.println("Thread is running...");  
    }  
    public static void main(String args[])  
    {  
        demoThread2 t1 = new demoThread2();  
        t1.start();  
    }  
}
```

Output:

Thread is running...

Example:

```
class ThreadDemo extends Thread  
{
```

```
Thread t;
String threadName;
ThreadDemo( String name)
{
    threadName = name;
    System.out.println("Creating " + threadName );
}
public void run()
{
    System.out.println("Running " + threadName );
    try
    {
        for(int i = 2; i >= 0; i--)
        {
            System.out.println("Thread: " + threadName + ", " + i);
            // Let the thread sleep for a while.
            Thread.sleep(1000);
        }
    }
    catch (InterruptedException e)
    {
        System.out.println("Thread " + threadName + " interrupted.");
    }
    System.out.println("Thread " + threadName + " exiting.");
}
public void start ()
{
    System.out.println("Starting " + threadName );
    if (t == null)
    {
        t = new Thread (this, threadName);
        t.start ();
    }
}
}
public class ThreadDemo2
{
    public static void main(String args[])
    {
        ThreadDemo T1 = new ThreadDemo( "Thread-1");
        T1.start();
    }
}
```

```
}
}
```

Output:

```
Creating Thread-1
Starting Thread-1
Running Thread-1
Thread: Thread-1, 2
Thread: Thread-1, 1
Thread: Thread-1, 0
Thread Thread-1 exiting.
```

Thread Synchronization (થ્રેડ સિંક્રોનાઈઝેશન)

- જ્યારે બે કે તેથી વધુ થ્રેડોને shared resource ને access કરવાની જરૂર હોય, ત્યારે એક સમયે ફક્ત એક જ થ્રેડ દ્વારા resource નો ઉપયોગ કરવામાં આવે, તે ensure કરવા માટે કેટલીક રીતની જરૂર હોય છે.
- તે process કે જેના દ્વારા આ સિંક્રોનાઈઝેશન કરવામાં આવે છે તેને થ્રેડ સિંક્રોનાઈઝેશન કહે છે
- જાવામાં synchronized કીવર્ડ થી કોડનો એક બ્લોક બનાવે છે, જેને critical section તરીકે ઓળખવામાં આવે છે.
- કોડના critical section માં દરેક જાવા object ને object સાથે સંકળાયેલ lock મળે છે.

Syntax:

```
synchronized(object)
{
    // statements to be synchronized
}
```

- અહીં, **object** એ **synchronized** થઈ રહેલા **object**નો **reference** છે.
- સિંક્રોનાઈઝડ **block** એ **ensures** કરે છે કે **current** થ્રેડ સફળતાપૂર્વક **object** ના **critical section**માં **enter** થયા પછી જ **object** ની **method** ને **call** કરી શકે છે.

Example:

```
class PrintDemo
{
    public void printCount()
    {
        try
        {
            for(int i = 3; i > 0; i--)
            {
                System.out.println("Counter --- " + i);
            }
        }
        catch (Exception e)
        {
            System.out.println("Thread interrupted.");
        }
    }
}

class ThreadDemo extends Thread
```

```
{
    Thread t;
    String threadName;
    PrintDemo PD;

    ThreadDemo( String name, PrintDemo pd)
    {
        threadName = name;
        PD = pd;
    }
    public void run()
    {
        synchronized(PD)
        {
            PD.printCount();
        }
        System.out.println("Thread " + threadName + " exiting.");
    }
    public void start ()
    {
        System.out.println("Starting " + threadName );
        if (t == null)
        {
            t = new Thread (this, threadName);
            t.start ();
        }
    }
}

public class ThreadSchro
{
    public static void main(String args[])
    {
        PrintDemo PD = new PrintDemo();
        ThreadDemo T1 = new ThreadDemo( "Thread - 1 ", PD );
        ThreadDemo T2 = new ThreadDemo( "Thread - 2 ", PD );

        T1.start();
        T2.start();
    }
}
```

Output:

```
Starting Thread - 1
Starting Thread - 2
Counter --- 3
Counter --- 2
Counter --- 1
```

Thread Thread - 1 exiting.

Counter --- 3

Counter --- 2

Counter --- 1

Thread Thread - 2 exiting.

Inter-thread Communication

- ઇન્ટર-થ્રેડ કમ્યુનિકેશન અથવા Co-operation, સિંક્રનાઈઝ થ્રેડોને એકબીજા સાથે કમ્યુનિકેશન કરવાની allow કરે છે
- ઇન્ટર-થ્રેડ કમ્યુનિકેશન એક મિકેનિઝમ છે જેમાં થ્રેડને તેના critical section માં paused કરવામાં આવે છે અને બીજા થ્રેડને એક્ઝિક્યુટ કરવા માટે same critical sectionમાં (અથવા lock) દાખલ કરવાની મંજૂરી છે.
- avoid polling(It is usually implemented by loop), ઇન્ટર-થ્રેડ communication એ Object class ની નીચેની method ઓ દ્વારા લાગુ કરવામાં આવે છે:
 - **wait()**: આ method calling થ્રેડને કહે છે કે critical section છોડી દો અને sleep થઈ જાઓ ત્યાં સુધી કેટલાક અન્ય થ્રેડ same critical sectionમાં આવે નહીં અને notify () method call કરે.
 - **notify()**: આ method પ્રથમ થ્રેડને wakes up કરે છે જેને same object પર wait() કરે છે.
 - **notifyAll()**: આ method તે બધા થ્રેડો ને wakes up કરે છે જેને same object પર wait () કરે છે. સૌથી વધુ priority હોય તે થ્રેડ પ્રથમ run થાય છે.
- ઉપરની બધી methods final તરીકે object class માં implement કરેલ છે.
- ત્રણેય methods ફક્ત synchronized context માંથી જ call કરી શકાય.

Example:

```
class Customer
{
    int amount=10000;
    synchronized void withdraw(int amount)
    {
        System.out.println("going to withdraw...");

        if(this.amount<amount)
        {
            System.out.println("Less balance; waiting for deposit...");
            try
            {
                wait();
            }
            catch(Exception e){}
        }
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }
    synchronized void deposit(int amount)
    {
        System.out.println("going to deposit...");
        this.amount+=amount;
    }
}
```

```
        System.out.println("deposit completed... ");
        notify();
    }
}
class InterThreadDemo
{
    public static void main(String args[])
    {
        final Customer c = new Customer();
        new Thread()
        {
            public void run(){c.withdraw(15000); }
        }.start();
        new Thread()
        {
            public void run(){c.deposit(10000);}
        }.start();
    }
}
```

Output:

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```